main.py

PYTHON PROGRAMMING ASSIGNMENT

Milk Payment Calculator
Tasks 1–12

Code-editor + output-panel PowerPoint layout

NAMES:B.MAHESH KUMAR(1925260023)

task1.py

Visualize

Run ▶

Output

Task 1 — Understand the Problem

```
 1  farmer_id = "F001"
 2  litres = 10.5
 3  fat = 4.2
 4  snf = 8.7
 5  shift = "Morning"
 6
 7  BASE_RATE = 30.00
 8  FAT_STANDARD = 3.5
 9  SNF_STANDARD = 8.5
10  FAT_STEP_VALUE = 0.40
11  SNF_STEP_VALUE = 0.30
12
13  if litres < 0:
14      print("Invalid litres")
15  elif not 2.0 <= fat <= 7.0:
16      print("Invalid fat")
17  else:
18      print("Farmer ID:", farmer_id)
19      print("Litres:", litres)
20      print("Fat:", fat)
21      print("SNF:", snf)
22      print("Shift:", shift)
23      print("Input is valid")
```

Program output

```
Farmer ID: F001
Litres: 10.5
Fat: 4.2
SNF: 8.7
Shift: Morning
Input is valid
```

What this demonstrates

Stores farmer identity, quantity, milk-quality readings, shift, and named rate constants. Validation rejects negative litres and fat outside the allowed range.

task2.py

Visualize

Run ▶

Output

Task 2 — Values and Types

```
 1  from decimal import Decimal
 2
 3  for value in [10, 10.0, "10", True, [10, 8.5]]:
 4      print(value, type(value))
 5
 6  print(7 / 2)
 7  print(7 // 2)
 8  print(7 % 2)
 9  print(-7 // 2)
10  print(0.1 + 0.2 == 0.3)
11
12  payments = ["33.40"] * 1000 + ["0.10"] * 3
13  total = sum(Decimal(x) for x in payments)
14  print("Decimal:", total)
15  print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Program output

```
10 <class 'int'>
10.0 <class 'float'>
10 <class 'str'>
True <class 'bool'>
[10, 8.5] <class 'list'>
3.5
3
1
-4
False
Decimal: 33400.30
True
```

What this demonstrates

Demonstrates Python types, floor division, modulo, floating-point representation, and exact decimal addition for money.

task3.py

Visualize

Run ▶

Output

Task 3 — Variables and Constants

```
1  BASE_RATE = 30.00
2  FAT_STANDARD = 3.5
3  SNF_STANDARD = 8.5
4  FAT_STEP_VALUE = 0.40
5  SNF_STEP_VALUE = 0.30
6
7  farmer_id, litres, fat = "F001", 10, 4.2
8  x = 10
9  print(type(x))
10 x = "Milk"
11 print(type(x))
12
13 a, b = 10, 20
14 a, b = b, a
15 print(a, b)
16
17 RATE_CARD = {
18     "base": 30.00,
19     "fat": 0.40,
20     "snf": 0.30
21 }
22 print(RATE_CARD)
```

Program output

```
<class 'int'>
<class 'str'>
20 10
{'base': 30.0, 'fat': 0.4, 'snf': 0.3}
```

What this demonstrates

UPPER_CASE communicates constants by convention. Python remains dynamically typed, so a variable can refer to an int and later a str.

task4.py

Visualize

Run ▶

Output

Task 4 — Correct Expressions

```python
import math

fat = 4.2
snf = 8.7
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)

rate = round(30 + fat_steps * 0.40
             + snf_steps * 0.30, 2)
amount = round(8.5 * rate, 2)

print(fat_steps, snf_steps, rate, amount)
print((4.2 - 3.5) / 0.1)
print(round(-3.7), int(-3.7), math.floor(-3.7))
print(2.0 <= fat <= 7.0)
```

Program output

```
7 2 33.4 283.9
7.000000000000001
-4 -3 -4
True
```

What this demonstrates

Shows binary float error, why rounded step counts are useful, negative-number behavior, and comparison chaining.

task5.py

Visualize

Run ▶

Output

Task 5 — Read Input and Convert Types

```
farmer_id = input("Farmer ID: ")

try:
    litres = float(input("Litres: "))
    fat = float(input("Fat %: "))
    snf = float(input("SNF %: "))
    shift = input("Shift: ").strip()

    if litres < 0:
        print("Invalid litres")
    elif not 2.0 <= fat <= 7.0:
        print("Invalid fat")
    elif snf < 0:
        print("Invalid SNF")
    else:
        print(f"{farmer_id:10} {litres:8.2f} "
              f"{fat:5.2f} {snf:5.2f} {shift}")
except ValueError:
    print("Invalid numeric input")
```

Program output

```
Farmer ID: F001
Litres: 10.5
Fat %: 4.2
SNF %: 8.7
Shift: Morning
F001          10.50  4.20  8.70 Morning
```

What this demonstrates

input() returns strings. float() converts numeric text; try/except catches values such as 'eight' and '8,5'.

task6.py

Visualize

Run ▶

Output

Task 6 — Organise the Logic into Functions

```
1  BASE_RATE = 30.00
2  FAT_STEP_VALUE = 0.40
3  SNF_STEP_VALUE = 0.30
4
5  def rate_per_litre(fat, snf):
6      """Return the rate per litre."""
7      fat_steps = round((fat - 3.5) / 0.1)
8      snf_steps = round((snf - 8.5) / 0.1)
9      return round(BASE_RATE + fat_steps * FAT_STEP_VALUE
10                  + snf_steps * SNF_STEP_VALUE, 2)
11
12 def payout(litres, fat, snf):
13     """Return the amount payable."""
14     return round(litres * rate_per_litre(fat, snf), 2)
15
16 def ten_day_total(payments):
17     """Return the total of payment amounts."""
18     return round(sum(payments), 2)
19
20 print(rate_per_litre(4.2, 8.7))
21 print(payout(8.5, 4.2, 8.7))
22 print(ten_day_total([300.00, 283.90, 333.60]))
```

Program output

```
33.4
283.9
917.5
```

What this demonstrates

Separates rate, payout, and total calculations. Functions return values so the results can be tested or reused without printing.

milkrate.py

Visualize

Run ▶

Output

Task 7 — Build a Module

```
 1  BASE_RATE = 30.00
 2  FAT_STEP_VALUE = 0.40
 3  SNF_STEP_VALUE = 0.30
 4
 5  def rate_per_litre(fat, snf):
 6      """Return rate per litre."""
 7      fs = round((fat - 3.5) / 0.1)
 8      ss = round((snf - 8.5) / 0.1)
 9      return round(BASE_RATE + fs * FAT_STEP_VALUE
10                   + ss * SNF_STEP_VALUE, 2)
11
12  def payout(litres, fat, snf):
13      """Return the payment amount."""
14      return round(litres * rate_per_litre(fat, snf), 2)
15
16  if __name__ == "__main__":
17      print(payout(8.5, 4.2, 8.7))
```

Program output

```
283.9
```

What this demonstrates

Save as milkrate.py. Importing the module defines its functions but does not run the demonstration block because __name__ is not '__main__'.

task8.py

Visualize

Run ▶

Output

Task 8 — Plan the Algorithm

```
 1  def calculate_payment(litres, fat, snf):
 2      if litres < 0 or not 2 <= fat <= 7 or snf < 0:
 3          raise ValueError("Invalid entry")
 4      fs = round((fat - 3.5) / 0.1)
 5      ss = round((snf - 8.5) / 0.1)
 6      rate = round(30 + fs * 0.40 + ss * 0.30, 2)
 7      return rate, round(litres * rate, 2)
 8
 9  def ten_day_total(entries):
10      total = 0.0
11      for litres, fat, snf in entries:
12          _, amount = calculate_payment(litres, fat, snf)
13          total += amount
14      return round(total, 2)
15
16  entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7),
17             (12, 3.1, 8.3)]
18  print(ten_day_total(entries))
```

Program output

```
917.5
```

What this demonstrates

Algorithm: validate an entry, count fat/SNF steps, compute rate and amount, add amount to the ten-day total. Running total uses O(1) extra space.

task9.py

Visualize

Run ▶

Output

Task 9 — Implement and Test

```
1  from decimal import Decimal
2
3  def rate_per_litre(fat, snf):
4      fs = round((float(fat) - 3.5) / 0.1)
5      ss = round((float(snf) - 8.5) / 0.1)
6      return (Decimal("30.00") + Decimal(fs) * Decimal("0.40")
7              + Decimal(ss) * Decimal("0.30")).quantize(
8                  Decimal("0.01"))
9
10 def payout(litres, fat, snf):
11     if litres < 0 or not 2 <= fat <= 7 or snf < 0:
12         raise ValueError("Invalid entry")
13     return (Decimal(str(litres)) * rate_per_litre(fat, snf)).quantize(
14         Decimal("0.01"))
15
16 assert payout(10, 3.5, 8.5) == Decimal("300.00")
17 assert payout(8.5, 4.2, 8.7) == Decimal("283.90")
18 assert payout(12, 3.1, 8.3) == Decimal("333.60")
19 print("Tests passed")
```

Program output

```
Tests passed
```

What this demonstrates

Assertions verify standard, above-standard, and below-standard calculations. Extend the suite with minimum/maximum fat and invalid inputs.

task10.py

Visualize

Run ▶

Output

Task 10 — Compare Expectations with Execution

```python
from decimal import Decimal
import time

def total_float(items):
    total = 0.0
    for x in items:
        total += float(x)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for x in items:
        total += Decimal(x)
    return total

for size in (300, 6000, 150000):
    payments = ["33.40"] * size
    start = time.perf_counter()
    f_total = total_float(payments)
    f_time = time.perf_counter() - start
    start = time.perf_counter()
    d_total = total_decimal(payments)
    d_time = time.perf_counter() - start
    print(size, f_total, d_total)
    print("seconds:", f_time, d_time)
```

Program output

```
300 10019.999999999995 10020.00
seconds: <measured> <measured>
6000 200400.0000000211 200400.00
seconds: <measured> <measured>
150000 5010000.000002312 5010000.00
seconds: <measured> <measured>
```

What this demonstrates

Float values shown are illustrative of typical binary rounding; run the code to obtain actual timings and exact observed float totals on your system.

task11.py

Visualize

Run ▶

Output

Task 11 — Recommend the Final Design

```python
from decimal import Decimal

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")

def rate_per_litre(fat, snf):
    fs = round((float(fat) - 3.5) / 0.1)
    ss = round((float(snf) - 8.5) / 0.1)
    return (BASE_RATE + Decimal(fs) * FAT_STEP_VALUE
            + Decimal(ss) * SNF_STEP_VALUE).quantize(
                Decimal("0.01"))

def payout(litres, fat, snf):
    if litres < 0 or not 2 <= fat <= 7 or snf < 0:
        raise ValueError("Invalid entry")
    return (Decimal(str(litres)) * rate_per_litre(fat, snf)).quantize(
        Decimal("0.01"))

print(payout(8.5, 4.2, 8.7))
```

Program output

```
283.90
```

What this demonstrates

Recommended design: central rate constants, validated input, Decimal or integer paise for money, automated tests, and a stored audit trail for union-scale use.

task12.py

Visualize

Run ▶

Output

Task 12 — Day-4 Design Challenge

```
 1  from decimal import Decimal
 2
 3  RATE_CARDS = {
 4      "regular": (Decimal("30.00"), Decimal("0.40"),
 5                  Decimal("0.30")),
 6      "seasonal": (Decimal("32.00"), Decimal("0.45"),
 7                   Decimal("0.30"))
 8  }
 9  BONUS_THRESHOLD = Decimal("500")
10  BONUS_PER_LITRE = Decimal("0.50")
11
12  def rate_per_litre(fat, snf, season):
13      base, fat_value, snf_value = RATE_CARDS[season]
14      fs = round((float(fat) - 3.5) / 0.1)
15      ss = round((float(snf) - 8.5) / 0.1)
16      return (base + Decimal(fs) * fat_value
17              + Decimal(ss) * snf_value).quantize(Decimal("0.01"))
18
19  def ten_day_payment(entries, season):
20      litres_total = sum(Decimal(str(e[0])) for e in entries)
21      milk_total = Decimal("0.00")
22      for litres, fat, snf in entries:
23          rate = rate_per_litre(fat, snf, season)
24          milk_total += (Decimal(str(litres)) * rate).quantize(
25              Decimal("0.01"))
26      bonus = litres_total * BONUS_PER_LITRE if litres_total > 500 else
Decimal("0.00")
27      return litres_total, milk_total, bonus, milk_total + bonus
```

Program output

```
Seasonal card: base ₹32.00
Fat step: ₹0.45
SNF step: ₹0.30
Bonus: ₹0.50/L only when total > 500 L
Use two-decimal analyser readings.
```

What this demonstrates

Retains the original SNF adjustment, adds seasonal rate selection, supports two-decimal analyser readings, and applies the bonus only when the ten-day total exceeds 500 litres.